

Object Oriented Programming with Java

25.3 Faculty of Computing



Take Home Activity 06

Answer all questions.

1. Create a class called **Device** with the following fields:
 - brand of type String
 - model of type String
 - isOn of type boolean
 - a. Add the following **methods** to Device:
 - turnOn() - sets isOn to true and prints: "[brand] [model] is now ON"
 - turnOff() - sets isOn to false and prints: "[brand] [model] is now OFF"
 - b. Create a **subclass** called **SmartDevice** that inherit to Device and adds:
 - appName of type String
 - connectToApp() - prints: "[brand] connected to [appName]"
 - c. In **main()**:
 - Create a SmartDevice object
 - Set brand to "Samsung", model to "Galaxy S24", appName to "SmartThings"
 - Call turnOn() (inherited)
 - Call connectToApp()
 - Call turnOff() (inherit)
2. Create a class called **NetworkDevice** with the following fields:
 - ipAddress of type String
 - macAddress of type String
 - a. Add the following methods to **NetworkDevice**:
 - connect() – prints: "Connecting device with IP: [ipAddress]"
 - disconnect() – prints: "Device [ipAddress] disconnected"
 - b. Create a **subclass** called **Firewall** that inherits to **NetworkDevice** and adds:
 - firewallName of type String
 - blockTraffic(String source) – prints: "[firewallName] blocked traffic from [source]"
 - allowTraffic(String source) – prints: "[firewallName] allowed traffic from [source]"
 - c. In **main()**:
 - Create a Firewall object

Set ipAddress to "192.168.1.1", macAddress to "AA:BB:CC:DD:EE:FF", firewallName to "PFSense"

Call connect() (inherited)

Call blockTraffic("10.0.0.5")

Call allowTraffic("10.0.0.9")

Call disconnect() (inherited)

3. Create a **class** called **Person** with the following fields:

name of type String

age of type int

Add a method introduce() that prints: "Hi, I am [name] and I am [age] years old."

a. Create a class called **Employee** that inherits to Person and adds:

employeeId of type String

department of type String

Add a method showEmployeeInfo() that prints: "Employee [employeeId] works in [department]"

b. Create a class called **SecurityAnalyst** that inherits Employee and adds:

clearanceLevel of type int

Add a method analyzeThreats() that prints: "Analyst [name] is analysing threats at clearance level [clearanceLevel]"

c. In **main()**:

Create a SecurityAnalyst object

Set name to "Diana", age to 29, employeeId to "SA-201", department to "SOC", clearanceLevel

to 3

Call introduce() (from Level 1)

Call showEmployeeInfo() (from Level 2)

Call analyzeThreats() (own method, Level 3) Create a Java class called

4. Create a class called **Vehicle** with the following fields:

make of type String

speed of type int

Add a method move() that prints: "[make] is moving at [speed] km/h"

a. Create a class called **ElectricVehicle** that inherits to Vehicle and adds:

batteryLevel of type int (percentage, 0–100)

Add a method chargeBattery() that sets batteryLevel to 100 and prints: "[make] battery fully charged."

Add a method showBattery() that prints: "Battery: [batteryLevel]%"

b. Create a class called **AutonomousVehicle** that inherits to ElectricVehicle and adds:

autopilotEnabled of type boolean

Add a method enableAutopilot() that sets autopilotEnabled to true and prints: "Autopilot ON for [make]"

c. In **main()**:

Create an AutonomousVehicle object
Set make to "Tesla", speed to 120, batteryLevel to 45
Call move() (Level 1)
Call showBattery() (Level 2)
Call chargeBattery() (Level 2)
Call enableAutopilot() (Level 3)

5. Create a class called Account with the following fields:

accountNumber of type String
holderName of type String
balance of type double

- a. Add the following methods to Account:

showDetails() – prints: "Account [accountNumber] | Holder: [holderName] | Balance: [balance]"
deposit(double amount) – adds amount to balance and prints: "Deposited [amount]. New balance: [balance]"

- b. Create three subclasses that all extend Account:

- a. SavingsAccount

- i. Adds field interestRate of type double
- ii. Adds method applyInterest() – increases balance by (balance * interestRate / 100) and prints: "Interest applied. New balance: [balance]"

- b. LoanAccount

- i. Adds field loanAmount of type double
- ii. Adds method repayLoan(double amount) – reduces loanAmount by amount and prints: "Loan repaid: [amount]. Remaining: [loanAmount]"

- c. FixedDepositAccount

- i. Adds field maturityDate of type String
- ii. Adds method showMaturity() – prints: "Fixed deposit matures on: [maturityDate]"

- c. In main():

Create one object of each subclass

For each: call showDetails() (inherited) and the subclass-specific method

6. Create a class called Threat with the following fields:

threatId of type String
source of type String
severity of type String (e.g., "Low", "Medium", "High", "Critical")

- a. Add the following methods to Threat:

logThreat() – prints: "[THREAT LOG] ID: [threatId] | Source: [source] | Severity: [severity]"

raiseAlert() – prints: "!! SECURITY ALERT !! Threat [threatId] detected from [source]"

b. Create three subclasses that all extend Threat:

- a. MalwareAttack
 - a. Adds field malwareType of type String (e.g., "Ransomware", "Trojan")
 - b. Adds method quarantine() – prints: "Quarantining [malwareType] from [source]..."
- b. PhishingAttack
 - a. Adds field targetEmail of type String
 - b. Adds method blockEmail() – prints: "Blocking phishing email targeting: [targetEmail]"
- c. DDoSAttack
 - a. Adds field packetCount of type int
 - b. Adds method mitigate() – prints: "Mitigating DDoS from [source] — [packetCount] packets dropped"
- d. In main():
 - a. Create one object of each subclass with appropriate values
 - b. For each object:
 - c. Call logThreat() (inherited)
 - d. Call raiseAlert() (inherited)
 - e. Call the subclass-specific method